

一、实验目的及要求

- 开始熟悉图形界面
- 熟悉枚举
- 熟悉继承
- 按照题目要求写代码和实验报告，并上传

二、实验题目及实现过程

1.设计选课系统

a) 学生分本科生（学号、姓名、班级）和研究生（学号、姓名、班级、导师）两种；

```
19     private final StudentLevel level; 4个用法
20     String id; 2个用法
21     String name; 3个用法
22     String klass; 2个用法
23     List<Course> courses = new ArrayList<>(); 2个用法
24
25     private Student(StudentLevel level,String id,String name,String klass){ 2个用法 新*
26         this.level = level;
27         this.id = id;
28         this.name = name;
29         this.klass = klass;
30     }
31
32     > public String level(){ return this.level.getDescription(); }
33
34     public Boolean isUndergraduate(){ return this.level == StudentLevel.Undergraduate; } 0个用法 新*
35     public Boolean isPostgraduate(){ return this.level == StudentLevel.Postgraduate; } 0个用法 新*
36
37     public String getInfoSafe(){ return "%s".formatted(level(),this.name); } 3个用法 新*
38     public String getInfoFull(){ return "%s(%s)班级%s".formatted(level(),this.name,this.id,this.klass); } 0个用法 新*
39
40     > public void addCourse(Course course) { courses.add(course); }
41
42
43     public static class Undergraduate extends Student { 1个用法 新*
44         public Undergraduate(String id, String name, String klass) { 1个用法 新*
45             super(StudentLevel.Undergraduate, id, name, klass);
46         }
47     }
48
49     static class Postgraduate extends Student { 1个用法 新*
50         String tutor; 1个用法
51
52         public Postgraduate(String id, String name, String klass, String tutor) { 1个用法 新*
53             super(StudentLevel.Postgraduate, id, name, klass);
54             this.tutor = tutor;
55         }
56     }
57
58 }
```

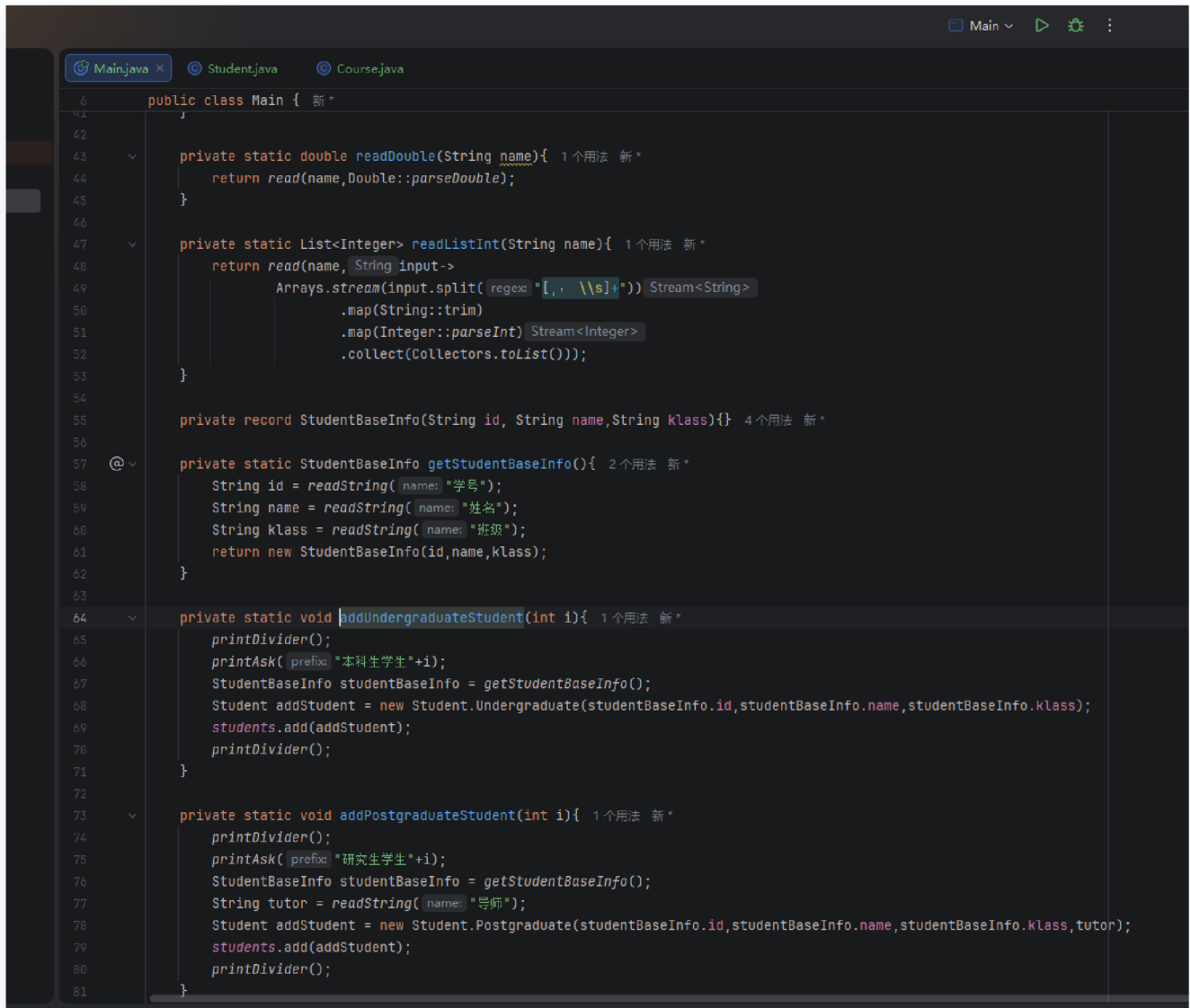
这段代码完成了学生类及其子类的基本结构设计。定义了一个基础的 `Student` 类，并在此基础上派生出 `Undergraduate`（本科生）和 `Postgraduate`（研究生）两个子类，分别包含了学号、姓名、班级以及研究生特有的导师等信息。同时，代码使用了枚举类型 `StudentLevel` 来标识学生的不同级别，提高了整体架构的规范性与可读性。

b) 课程（编号、课程名、学分）分必修和选修两种；

```
1 public class Course { 16个用法 2个继承者 新*
2     private enum CourseType { 6个用法 新*
6         private final String description; 2个用法
7
8     >     CourseType(String description){ this.description = description; }
11
12     public String getDescription() { 1个用法 新*
13         return this.description;
14     }
15 }
16
17     private final CourseType type; 4个用法
18     public String id; 2个用法
19     public String name;
20     public double credit; 2个用法
21
22     private Course(CourseType type,String id,String name,double credit){ 2个用法 新*
23         this.type = type;
24         this.id = id;
25         this.name = name;
26         this.credit = credit;
27     }
28
29     public String type(){ return this.type.getDescription(); } 2个用法 新*
30
31     public Boolean isCompulsory(){ return this.type == CourseType.Compulsory; } 1个用法 新*
32     public Boolean isElective(){ return this.type == CourseType.Elective; } 1个用法 新*
33
34     public String getInfoSafe(){ return "%s%s".formatted(type(),this.name); } 2个用法 新*
35     public String getInfoFull(){ return "%s%s(%s)学分%.2f".formatted( ...args: type(),this.name,this.id,this.credit); } 1个用法 新*
36
37     static class Compulsory extends Course{ 1个用法 新*
38     >     public Compulsory(String id,String name,double credit){ super(CourseType.Compulsory,id,name,credit); }
41 }
42
43     static class Elective extends Course{ 1个用法 新*
44     >     public Elective(String id,String name,double credit){ super(CourseType.Elective,id,name,credit); }
47 }
48 }
49 }
```

随后，我完成了课程类的面向对象设计。构建了基类 `Course` 并派生出 `Compulsory`（必修课）和 `Elective`（选修课）两个子类，其属性涵盖了课程编号、课程名和学分。代码中引入了 `CourseType` 枚举来区分具体的课程类型，并提供了相应的属性获取和格式化输出方法，以供后续系统进行统一调用与展示。

c) 创建4个学生信息（2个本科生， 2个研究生）



```
6 public class Main { 新 *
42 }
43 private static double readDouble(String name){ 1个用法 新 *
44     return read(name,Double::parseDouble);
45 }
46
47 private static List<Integer> readListInt(String name){ 1个用法 新 *
48     return read(name, String input->
49         Arrays.stream(input.split(regex: "[, \\s]+")) Stream<String>
50             .map(String::trim)
51             .map(Integer::parseInt) Stream<Integer>
52             .collect(Collectors.toList()));
53 }
54
55 private record StudentBaseInfo(String id, String name,String klass){} 4个用法 新 *
56
57 private static StudentBaseInfo getStudentBaseInfo(){ 2个用法 新 *
58     String id = readString( name: "学号");
59     String name = readString( name: "姓名");
60     String klass = readString( name: "班级");
61     return new StudentBaseInfo(id,name,klass);
62 }
63
64 private static void addUndergraduateStudent(int i){ 1个用法 新 *
65     printDivider();
66     printAsk( prefix: "本科生学生"+i);
67     StudentBaseInfo studentBaseInfo = getStudentBaseInfo();
68     Student addStudent = new Student.Undergraduate(studentBaseInfo.id,studentBaseInfo.name,studentBaseInfo.klass);
69     students.add(addStudent);
70     printDivider();
71 }
72
73 private static void addPostgraduateStudent(int i){ 1个用法 新 *
74     printDivider();
75     printAsk( prefix: "研究生学生"+i);
76     StudentBaseInfo studentBaseInfo = getStudentBaseInfo();
77     String tutor = readString( name: "导师");
78     Student addStudent = new Student.Postgraduate(studentBaseInfo.id,studentBaseInfo.name,studentBaseInfo.klass,tutor);
79     students.add(addStudent);
80     printDivider();
81 }
```

为了实例化测试数据，这部分代码实现了控制台信息的交互与录入。通过定义 `getStudentBaseInfo` 等辅助读取方法，程序能够提示用户逐一输入本科生和研究生的详细信息，并实例化对应的对象存入全局的学生列表中，从而成功创建了4名学生的初始化信息。

d) 创建4门课程信息（2门必修，2门选修）

```
6      public class Main { 新 *
71      }
72
73      private static void addPostgraduateStudent(int i){ 1个用法 新 *
74          printDivider();
75          printAsk( prefix: "研究生学生"+i);
76          StudentBaseInfo studentBaseInfo = getStudentBaseInfo();
77          String tutor = readString( name: "导师");
78          Student addStudent = new Student.Postgraduate(studentBaseInfo.id,studentBaseInfo.name,studentBaseInfo.klass,tutor);
79          students.add(addStudent);
80          printDivider();
81      }
82
83      private record CourseBaseInfo(String id,String name,double credit){} 4个用法 新 *
84
85      @ private static CourseBaseInfo getCourseBaseInfo(){ 2个用法 新 *
86          String id = readString( name: "编号");
87          String name = readString( name: "课程名");
88          double credit = readDouble( name: "学分");
89          return new CourseBaseInfo(id,name,credit);
90      }
91
92      private static void addCompulsoryCourse(int i){ 1个用法 新 *
93          printDivider();
94          printAsk( prefix: "必修课程"+i);
95          CourseBaseInfo courseBaseInfo = getCourseBaseInfo();
96          Course addCourse = new Course.Compulsory(courseBaseInfo.id,courseBaseInfo.name,courseBaseInfo.credit);
97          courses.add(addCourse);
98          printDivider();
99      }
100
101      private static void addElectiveCourse(int i){ 1个用法 新 *
102          printDivider();
103          printAsk( prefix: "选修课程"+i);
104          CourseBaseInfo courseBaseInfo = getCourseBaseInfo();
105          Course addCourse = new Course.Elective(courseBaseInfo.id,courseBaseInfo.name,courseBaseInfo.credit);
106          courses.add(addCourse);
107          printDivider();
108      }
109  }
```

与学生信息的录入过程类似，此模块代码实现了课程信息的动态创建。通过统一读取课程的编号、名称和学分等基础属性，分别调用并执行必修课与选修课的构建逻辑，完成了4门课程（2门必修，2门选修）对象的实例化与列表存储。

e) 自动选课部分：为每个学生自动选修所有必修课；

```
101 private static void addElectiveCourse(int i){ 1个用法 新 *
102     printDivider();
103     printAsk( prefix: "选修课程"+i);
104     CourseBaseInfo courseBaseInfo = getCourseBaseInfo();
105     Course addCourse = new Course.Elective(courseBaseInfo.id,courseBaseInfo.name,courseBaseInfo.credit);
106     courses.add(addCourse);
107     printDivider();
108 }
109
110 @ private static void printAddedAction(Student student,Course course){ 1个用法 新 *
111     System.out.printf("已为%s成功添加%s\n",student.getInfoSafe(),course.getInfoSafe());
112 }
113
114 private static void autoStudentSelectCompulsoryCourse(){ 1个用法 新 *
115     printDivider();
116     System.out.println("自动为每个学生选修所有必修课");
117     List<Course> compulsoryCourses = new ArrayList<>();
118     for(Course course: courses){
119         if(course.isCompulsory()){
120             compulsoryCourses.add(course);
121         }
122     }
123     for(Student student:students){
124         for(Course course: compulsoryCourses){
125             student.addCourse(course);
126             printAddedAction(student,course);
127         }
128     }
129     printDivider();
130 }
131
```

本段代码核心实现了系统要求的自动选课功能模块。程序首先遍历所有已创建的课程并筛选出必修课集合，随后遍历所有的学生对象，自动为每一位学生分配并添加所有必修课程，最后在控制台打印出选课成功的操作日志，确保无遗漏。

f) 秘书手动选课部分：为每个同学选修1-2门选修课；

```
133 @ private static void AskCourseAdded(Student student,List<Course> courses){ 1个用法 新 *
134     String hint = IntStream.range(0,courses.size()) IntStream
135         .mapToObj( int i->"%d为%s".formatted( ...args: i,courses.get(i).getInfoSafe())) Stream<String>
136         .collect(Collectors.joining( delimiter: ", "));
137     List<Integer> courseIds = readListInt("%s的选课信息(%s)用逗号分割".formatted(student.getInfoSafe(),hint));
138     for(Integer courseId: courseIds){
139         student.addCourse(courses.get(courseId));
140     }
141 }
142
143 private static void manualStudentSelectElectiveCourse(){ 1个用法 新 *
144     printDivider();
145     System.out.println("请教学秘书手动进行选修课的输入");
146     List<Course> electiveCourses = new ArrayList<>();
147     for(Course course: courses){
148         if(course.isElective()){
149             electiveCourses.add(course);
150         }
151     }
152     for(Student student:students){
153         AskCourseAdded(student,electiveCourses);
154     }
155     printDivider();
156 }
```

此部分代码展示了教学秘书进行手动选课的具体实现流程。程序先单独筛选出所有的选修课，并提示秘书为每位同学通过输入序号的方式选择所需的课程（按照要求每位同学选修1-2门）。系统读取输入的编号后，准确将选修课分配至相应学生名下。

g) 打印出每个学生的选课信息

```
157  @  private static void printFullStudentData(Student student){ 1个用法 新 *
158      String courseInfo = student.courses.stream() Stream<Course>
159          .map( Course course -> "%s".formatted(course.getInfoFull())) Stream<String>
160          .collect(Collectors.joining( delimiter: ", "));
161      System.out.printf("%s的选课信息: %s\n", student.getInfoSafe(), courseInfo);
162  }
163
164  private static void printAllStudentInfo(){ 1个用法 新 *
165      printDivider();
166      for(Student student:students){
167          printFullStudentData(student);
168      }
169      printDivider();
170  }
171
172  public static void main(String[] args) { 1个用法 新 *
```

在选课流程结束后，该方法负责格式化输出选课的最终结果。借助 Java Stream API 对每位学生名下的课程列表进行映射和字符串拼接，程序能够在控制台上清晰、直观地打印出每一位学生的个人基本信息及其对应已选修的全部必修和选修课程明细。

2.写一个有理数类

a) Rational 类的定义

```

1 import java.math.*;
2
3 public class Rational extends Number implements Comparable<Rational> { 222 个用法 新 *
4     //定义内部数据结构
5     > protected enum NumberType{...}
21     private BigInteger numerator = BigInteger.ZERO; 26 个用法
22     private BigInteger denominator = BigInteger.ONE; 28 个用法
23     private NumberType type = NumberType.Zero; 57 个用法
24     > protected NumberType numberType_from_signNum(int t){...}
29     protected Rational(){ 6 个用法 新 *
30     > protected Rational(BigInteger numerator, BigInteger denominator, NumberType type){...}
36     > protected void from_NotANumber(){...}
42     > protected void from_PositiveInfinity(){...}
48     > protected void from_NegativeInfinity(){...}
54     > protected void from_integer(BigInteger num){...}
60     > protected void from_fraction(BigInteger numerator, BigInteger denominator){...}
66     > protected void from_string(String num){...}
84     > protected void from_decimal(BigDecimal bd){...}
99     > protected void from_double(double num){...}
112     > protected static Rational anyToRational(Object obj) {...}
126     //常用的数字
127     public static final Rational ZERO = new Rational(BigInteger.ZERO, BigInteger.ONE, NumberType.Zero); 4 个用法
128     public static final Rational ONE = new Rational(BigInteger.ONE, BigInteger.ONE, NumberType.Positive); 1 个用法
129     public static final Rational TWO = new Rational(BigInteger.TWO, BigInteger.ONE, NumberType.Positive); 0 个用法
130     public static final Rational NEGATIVE_ONE = new Rational(BigInteger.ONE, BigInteger.ONE, NumberType.Negative); 1 个用法
131     public static final Rational TEN = new Rational(BigInteger.TEN, BigInteger.ONE, NumberType.Positive); 0 个用法
132     protected static final Rational NOT_A_NUMBER = new Rational(BigInteger.ZERO, BigInteger.ZERO, NumberType.NotANumber); 8 个用法
133     protected static final Rational POSITIVE_INFINITY = new Rational(BigInteger.ONE, BigInteger.ZERO, NumberType.PositiveInfinity); 7 个用法
134     protected static final Rational NEGATIVE_INFINITY = new Rational(BigInteger.ONE, BigInteger.ZERO, NumberType.NegativeInfinity); 5 个用法
135     //对外暴露接口创建类
136     > public static Rational of(long num){ return Rational.of(BigInteger.valueOf(num)); }
139     > public static Rational of(BigInteger num){...}
144     > public static Rational of(double num){...}
149     > public static Rational of(BigDecimal num){...}
154     > public static Rational of(String num){...}
159     > public static Rational of(Rational num){ return num; }
162     > public static Rational of(BigInteger n, double d){ return Rational.of(Rational.of(n), Rational.of(d)); }
165     > public static Rational of(BigInteger n, BigDecimal d){ return Rational.of(Rational.of(n), Rational.of(d)); }
168     > public static Rational of(BigInteger n, String d){ return Rational.of(Rational.of(n), Rational.of(d)); }
171     > public static Rational of(BigInteger n, BigInteger d){ return Rational.of(Rational.of(n), Rational.of(d)); }
174     > public static Rational of(BigInteger n, Rational d){ return Rational.of(Rational.of(n), Rational.of(d)); }

```

```

177     > public static Rational of(double n, long d){ return Rational.of(Rational.of(n), Rational.of(d)); }
180     > public static Rational of(double n, BigInteger d){ return Rational.of(Rational.of(n), Rational.of(d)); }
183     > public static Rational of(double n, double d){ return Rational.of(Rational.of(n), Rational.of(d)); }
189     > public static Rational of(double n, BigDecimal d){ return Rational.of(Rational.of(n), Rational.of(d)); }
189     > public static Rational of(double n, String d){ return Rational.of(Rational.of(n), Rational.of(d)); }
192     > public static Rational of(double n, Rational d){ return Rational.of(Rational.of(n), Rational.of(d)); }
195     > public static Rational of(BigDecimal n, long d){ return Rational.of(Rational.of(n), Rational.of(d)); }
198     > public static Rational of(BigDecimal n, BigInteger d){ return Rational.of(Rational.of(n), Rational.of(d)); }
201     > public static Rational of(BigDecimal n, double d){ return Rational.of(Rational.of(n), Rational.of(d)); }
204     > public static Rational of(BigDecimal n, BigDecimal d){ return Rational.of(Rational.of(n), Rational.of(d)); }
207     > public static Rational of(BigDecimal n, String d){ return Rational.of(Rational.of(n), Rational.of(d)); }
210     > public static Rational of(BigDecimal n, Rational d){ return Rational.of(Rational.of(n), Rational.of(d)); }
213     > public static Rational of(String n, long d){ return Rational.of(Rational.of(n), Rational.of(d)); }
216     > public static Rational of(String n, BigInteger d){ return Rational.of(Rational.of(n), Rational.of(d)); }
219     > public static Rational of(String n, double d){ return Rational.of(Rational.of(n), Rational.of(d)); }
222     > public static Rational of(String n, BigDecimal d){ return Rational.of(Rational.of(n), Rational.of(d)); }
225     > public static Rational of(String n, String d){ return Rational.of(Rational.of(n), Rational.of(d)); }
228     > public static Rational of(String n, Rational d){ return Rational.of(Rational.of(n), Rational.of(d)); }
231     > public static Rational of(Rational n, long d){ return Rational.of(Rational.of(n), Rational.of(d)); }
234     > public static Rational of(Rational n, BigInteger d){ return Rational.of(Rational.of(n), Rational.of(d)); }
237     > public static Rational of(Rational n, double d){ return Rational.of(Rational.of(n), Rational.of(d)); }
240     > public static Rational of(Rational n, BigDecimal d){ return Rational.of(Rational.of(n), Rational.of(d)); }
243     > public static Rational of(Rational n, String d){ return Rational.of(Rational.of(n), Rational.of(d)); }
246     > public static Rational of(Rational n, Rational d){ return n.divide(d); }
249     > protected synchronized void preThings(){...}
250     //字段访问器
259     > public BigInteger getNumerator(){ return this.numerator; }
262     > public BigInteger getDenominator(){ return this.denominator; }
265     //toString方法
266     > private String toStringPositive(){...}
273     @Override 新 *
274     > public String toString(){...}
284     //基本运算方法
285     > public Rational negate(){...}
295     > public Rational reciprocal(){...}
303     > public <T> Rational add(T r){ return this.add(Rational.anyToRational(r)); }
306     > public Rational add(Rational r){...}
337     > public <T> Rational subtract(T r){ return this.subtract(Rational.anyToRational(r)); }
340     > public Rational subtract(Rational r){ return this.add(r.negate()); }

```

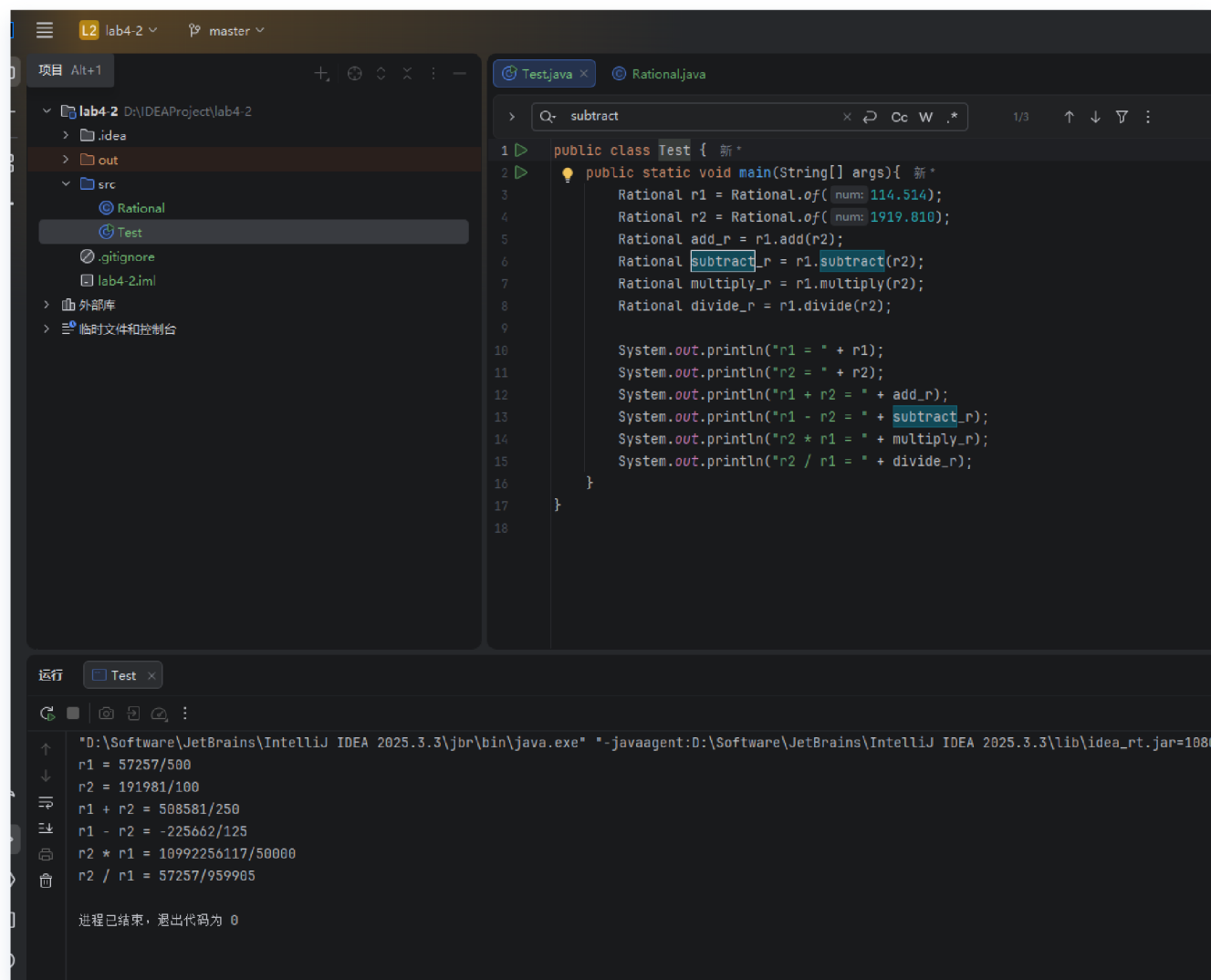
```

543 > public <T> Rational multiply(T r) { return this.multiply(Rational.anyToRational(r)); }
546 > public Rational multiply(Rational r){...}
577 > public <T> Rational divide(T r) { return this.divide(Rational.anyToRational(r)); }
580 @ > public Rational divide(Rational r) { return this.multiply(r.reciprocal()); }
583 > public Rational abs(){...}
591 > public int signNum(){...}
599 > public BigInteger bigIntegerSignNum(){...}
607 > public Rational rationalSignNum(){...}
617 // equals方法
618 @Override 7个用法 新+
619 > public boolean equals(Object o){...}
625 // Comparable接口
626 @Override 1个用法 新+
627 > public int compareTo(Rational r){...}
635 //进制转换
636 @ > private BigDecimal divideBigDecimalUnsigned(MathContext mc){...}
641 @ > private BigInteger divideBigIntegerUnsigned() { return this.numerator.divide(this.denominator); }
644 @Override 3个用法 新+
645 > public double doubleValue(){...}
675 @Override 2个用法 新+
676 > public float floatValue(){...}
686 @Override 2个用法 新+
687 > public int intValue(){...}
696 @Override 3个用法 新+
697 > public long longValue(){...}
706 > public BigDecimal bigDecimalValue(MathContext mc){...}
714 > public BigInteger bigIntegerValue(){...}
722 // hashCode接口
723 @Override 0个用法 新+
724 > public int hashCode() {...}
730 }
731

```

针对第二个实验任务，我设计并实现了一个功能高度完备的有理数类 `Rational`。该类继承自 `Number` 根类并实现了 `Comparable` 接口，内部采用 `BigInteger` 存储分子和分母以确保计算的极高精度，同时重载了加、减、乘、除等四则运算方法，并编写了大量支持多种数据类型转换的静态工厂方法。

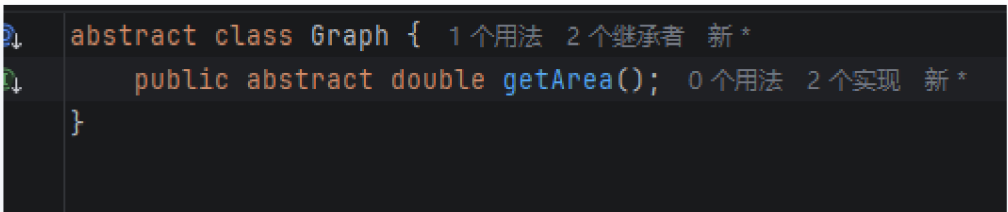
b) 测试



为了验证有理数运算逻辑的正确性，我编写了测试主类并进行了四则运算实例测试。运行结果清晰显示，程序能够正确无误地完成两个有理数对象的加法、减法、乘法和除法运算，并以标准的分数形式输出了精确的计算结果，完美符合实验对有理数精确计算的预期要求。

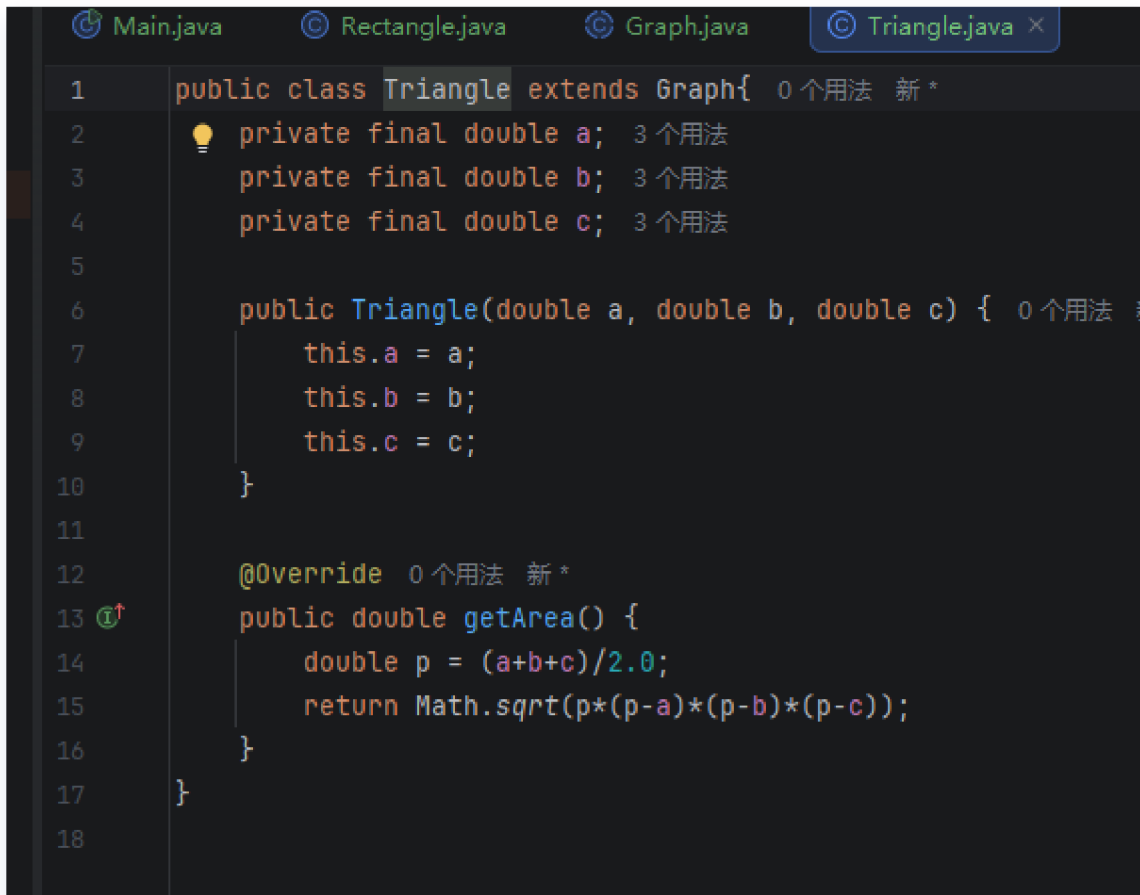
3.实现一个基础图形类Graph

a) 基础图形类Graph



在第三个基础图形类的任务中，我首先构建了一个基础的抽象类 `Graph`。该抽象类作为具体图形的父类，内部声明了一个抽象方法 `getArea()`，用于规范并要求所有继承该类的子图形（如矩形和三角形）必须具体实现各自的面积计算逻辑，充分运用了面向对象的抽象与多态特性。

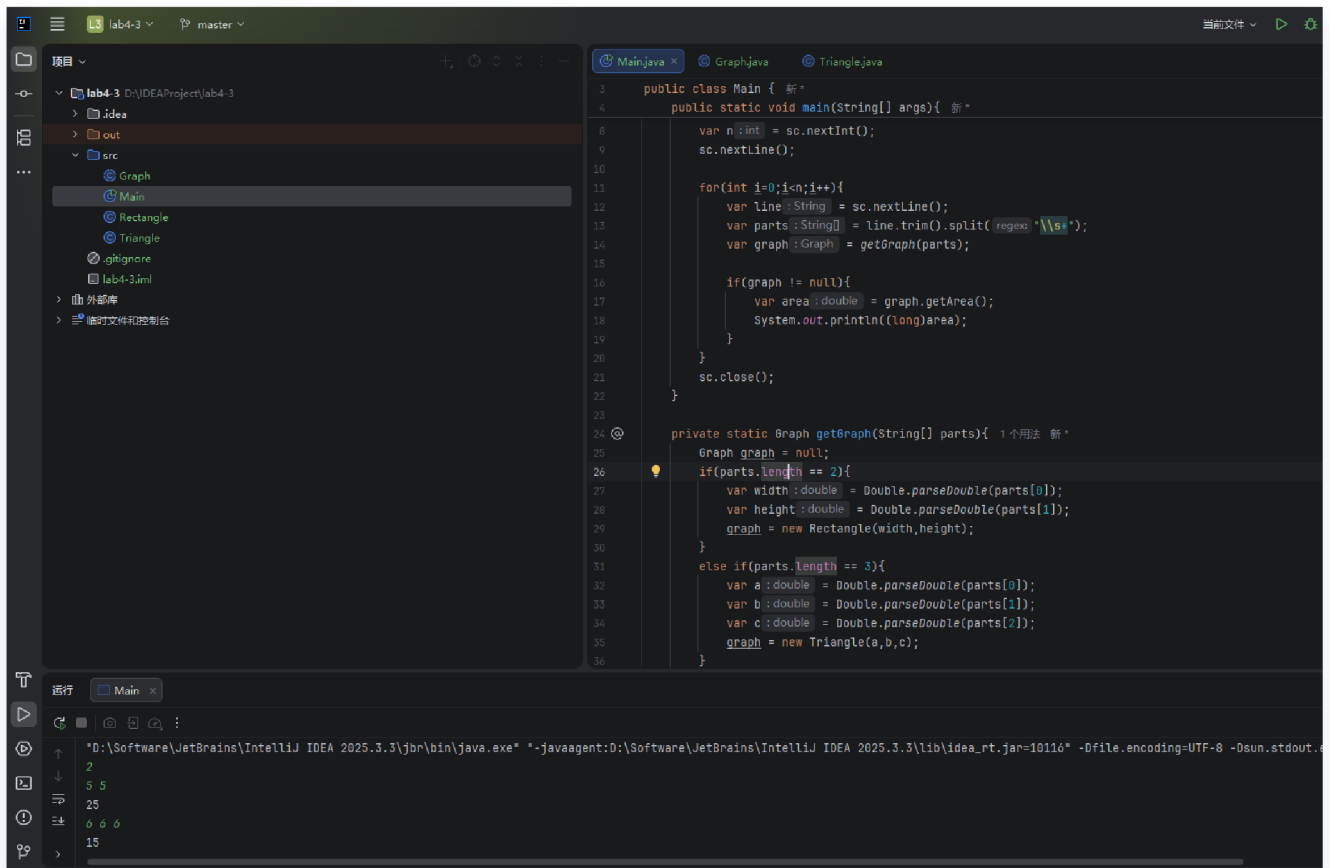
b) 三角形类 Triangle



```
1 public class Triangle extends Graph { 0 个用法 新 *
2     private final double a; 3 个用法
3     private final double b; 3 个用法
4     private final double c; 3 个用法
5
6     public Triangle(double a, double b, double c) { 0 个用法 新 *
7         this.a = a;
8         this.b = b;
9         this.c = c;
10    }
11
12    @Override 0 个用法 新 *
13    public double getArea() {
14        double p = (a+b+c)/2.0;
15        return Math.sqrt(p*(p-a)*(p-b)*(p-c));
16    }
17 }
18
```

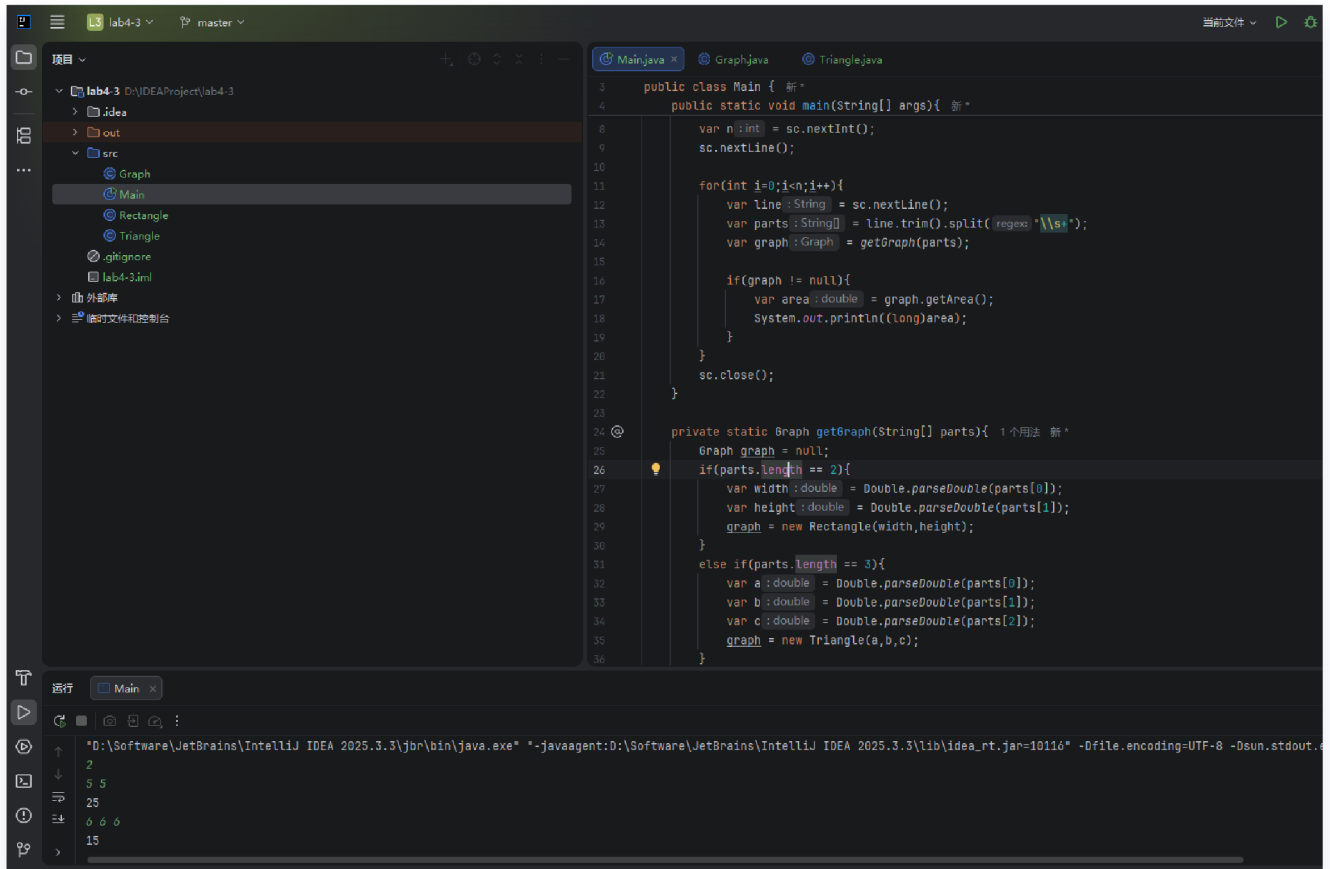
依托于 `Graph` 抽象父类，我完成了三角形子类 `Triangle` 的具体实现。该类封装了三个私有属性以代表三角形的三条边，并严格重写了 `getArea()` 方法，利用海伦公式（其中 $p = (a + b + c)/2$ ，面积为 $\sqrt{p(p-a)(p-b)(p-c)}$ ）精确计算并返回该三角形的真实面积。

c) 矩形类 Rectangle



同样地，矩形类 `Rectangle` 亦继承自 `Graph`，具备长和宽属性并实现了相应的矩形面积计算逻辑。配合主控制台程序的输入解析模块，系统能够利用空格分割用户输入的字符串，并根据具体参数个数（两个参数对应矩形，三个参数对应三角形）动态实例化对应的图形子类对象。

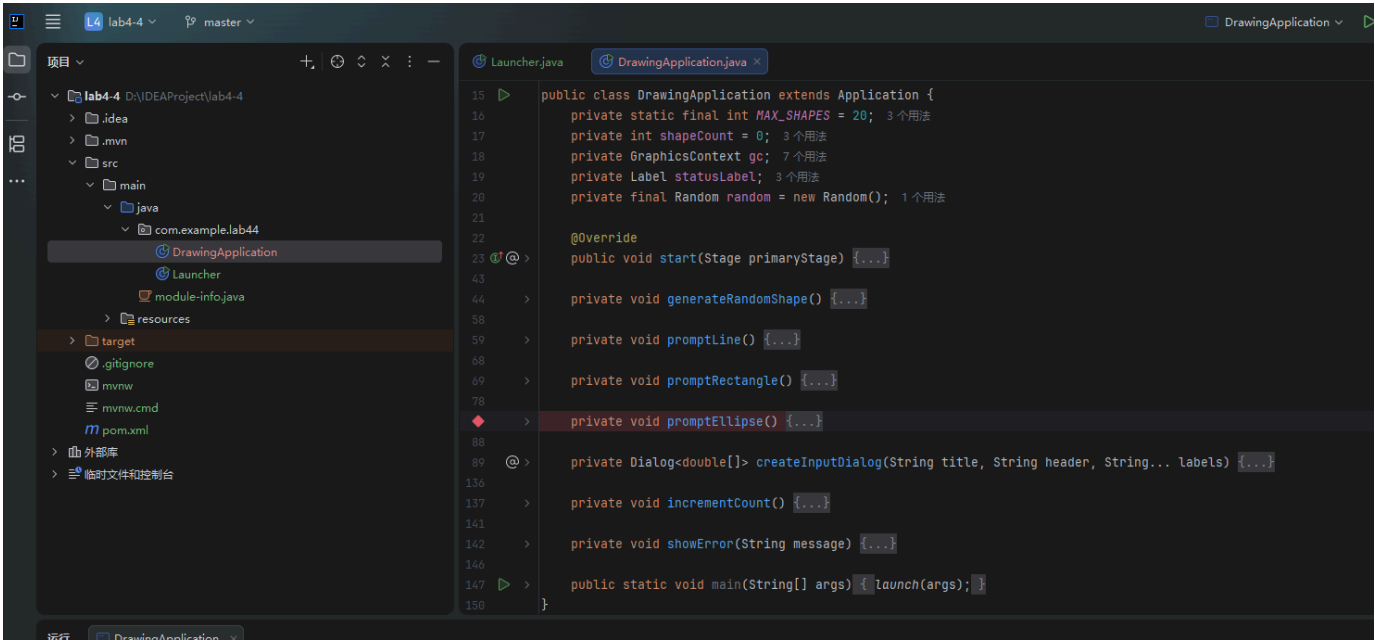
d) 测试



上图展示了基础图形类的最终控制台运行与测试结果。通过在控制台输入实验要求的测试样例（例如给定两个边长参数 5 5，或者三个边长参数 6 6 6），程序能够根据输入参数动态创建对应的矩形或三角形，并成功计算输出其面积值（分别为25和15），充分验证了类的继承和面积多态计算方法的准确性。

。

4. 创建一个简单的JavaFX绘图程序



第四个实验任务是开发基于 JavaFX 的简单绘图应用程序。代码中构建了核心业务逻辑，利用随机数生成机制决定接下来将要绘制的是直线、矩形还是椭圆，并预留了用户输入提示功能以获取图形的关键坐标与尺寸参数。同时，程序架构中已规划了参数合法性检查以及最多创建20个图形的总体限制要求。

5. 写一个交通信号灯枚举类TrafficLight

a) TrafficLight 类的定义

```

1 public enum TrafficLight { 0 个用法 新 *
2     RED( name: "红灯", r: 255, g: 0, b: 0), 0 个用法
3     YELLOW( name: "黄灯", r: 255, g: 255, b: 0), 0 个用法
4     GREEN( name: "绿灯", r: 0, g: 255, b: 0); 0 个用法
5
6     public record RGB(int r, int g, int b) {} 3 个用法 新 *
7
8     private final RGB rgb; 5 个用法
9     private final String name; 2 个用法
10    TrafficLight(String name,int r,int g,int b){ 6 个用法 新 *
11        this.rgb = new RGB(r,g,b);
12        this.name = name;
13    }
14    public int getR(){return this.rgb.r;} 0 个用法 新 *
15    public int getG(){return this.rgb.g;} 0 个用法 新 *
16    public int getB(){return this.rgb.b;} 0 个用法 新 *
17    public RGB getRGB(){return this.rgb;} 0 个用法 新 *
18    public String getName(){return this.name;} 0 个用法 新 *
19 }
20

```

在最后一个枚举实验任务中，我完整定义了交通信号灯枚举类 `TrafficLight`。该枚举类规范地声明了 `RED`（红灯）、`YELLOW`（黄灯）和 `GREEN`（绿灯）三个枚举常量，并利用嵌套的 `RGB` 记录类（record）为每一种灯光状态封装了对应的红、绿、蓝色彩通道数值及其中文名称描述。

b) 测试

```

1 public class Test { 新 *
2 @ private static void printColor(TrafficLight trafficLight) { 3 个用法 新 *
3     System.out.printf("%s的RGB值为(%d,%d,%d)\n",trafficLight.getName(),trafficLight.getR(),trafficLight.getG(),trafficLight.getB());
4 }
5
6 public static void main(String[] args) { 新 *
7     printColor(TrafficLight.GREEN);
8     printColor(TrafficLight.YELLOW);
9     printColor(TrafficLight.RED);
10 }
11 }
12

```

最终，我编写了交通信号灯测试类对枚举属性进行了遍历输出测试。通过调用专门的 `printColor` 方法以及枚举常量的各项 Getter 访问器方法（如 `getR()`、`getG()` 等），控制台成功打印出了红、黄、绿三种交通信号灯颜色的具体 RGB 组成数值，圆满达成了对枚举类特性的实践要求。

三、实验总结与心得记录

本次实验内容丰富，涵盖了类的继承、多态、枚举、高精度数值计算以及基础的JavaFX图形界面开发。通过独立完成这五个模块的编码与调试，我不仅巩固了Java语言的基础语法，更对面向对象系统设计有了更为深刻的工程化理解。

首先，在“选课系统”与“基础图形类”的设计中，我深刻体会到了继承与多态在代码复用和架构扩展上的优势。作为软件工程专业的学生，我逐渐意识到，相较于我以往更关注的底层C++那种追求极致运行速度和内存控制的开发模式，Java的纯面向对象范式更侧重于业务逻辑的抽象与规范。通过提取基类（如 `Graph` 或 `Course`）并在子类中重写抽象方法（如 `getArea()`），代码的耦合度显著降低，这为未来构建大规模、高扩展性的应用系统奠定了思维基础。

其次，在编写有理数类（`Rational`）的过程中，为了满足分数精确计算的需求，我使用了 `BigInteger` 来替代传统的浮点数类型。这让我认识到在不同应用场景下，性能与精度之间的权衡。虽然大数类的计算开销高于原生数据类型，但在需要绝对精度的场景下，这种设计是不可或缺的。同时，将有理数类作为 `Number` 的子类并实现 `Comparable` 接口，也让我进一步熟悉了Java标准库的接口规范与泛型设计。

此外，JavaFX绘图程序的实现是我初步涉足Java原生桌面UI开发的尝试。JavaFX提供了一套更为成熟且高度封装的事件驱动模型与组件库。这极大地简化了图形的绘制与用户交互逻辑的处理，但也要求开发者必须严格遵循其界面的生命周期与参数校验机制。

最后，交通信号灯枚举类（`TrafficLight`）的编写让我见识到了Java枚举的强大之处。结合 `Record` 语法，枚举不仅能表示状态常量，还能携带丰富的属性（如RGB值）和方法。这种强类型且自带封装属性的设计，大大提升了代码的安全性和表现力。

总体而言，本次实验不仅提升了我的Java编码熟练度，更促使我完成了从“关注底层实现”到“关注顶层对象抽象”的编程思维转换。在未来的学习与工程实践中，我将继续探索如何将严谨的面向对象设计与高效的代码执行逻辑更好地结合起来。